

There are many deep learning libraries available. You've probably heard about: **Keras, TensorFlow, and Darknet**.

Here we will use **Darknet** because it has implemented YOLOv3, which is an object detection model. It's really fast, especially when using it in a CUDA-capable system. It's meant to do real-time detections, which for robotics is quite important.



3.4 2D YOLO Detections

We'll begin by creating our own launches that use a version of **YOLO** adapted for ROS:

Execute in Shell #1

```
roscd my_object_recognition_pkg
touch launch/yolo_v2_tiny.launch
```

In []:

- yolo_v2_tiny.launch -

In []:

```
<?xml version="1.0" encoding="utf-8"?>

<launch>

  <!-- Use YOLOv3 -->
  <arg name="network_param_file"          default="$(find
darknet_ros)/config/yolov2-tiny.yaml"/>
WHAT IS THIS FILE ???
  <arg name="image" default="/camera/rgb/image_raw" />

  <!-- Include main launch file -->
  <include file="$(find darknet_ros)/launch/darknet_ros.launch">
WHAT IS THIS FILE ???

    <arg name="network_param_file"      value="$(arg network_param_file)"/>
    <arg name="image" value="$(arg image)" />
  </include>

</launch>
```

- yolo_v2_tiny.launch -

That simple. So let's have a look at some of the parameters:

In []:

```
<arg name="image" default="/camera/rgb/image_raw" />
```

Set the input **RGB ROS camera topic**. It's the most important and the only parameter you will need to change.

WHAT TOPIC DO I HAVE

In []:

```
<arg name="network_param_file" default="$(find  
darknet_ros)/config/yolov3.yaml"/>
```

There are different versions of **YOLO**: v2, v3, and v4. Each one works with different OpenCV libraries and works, in theory, better with each new version.

In this unit, we will use **v2 Tiny** (a faster version) because of the low impact on the system and fast performance.

The rest of the code is installed already in the system.

What is different between each file ??? YAML

If you want to access this version, you can download it from our Git: [YOLO ROS TheConstruct](#). If you want the original source, have a look at the following Gits:

READ THESE

- [YOLO Darknet ROS](#)
- [Darknet ROS](#)
- [Original Source Git](#)

Start the launch and see the output:

Execute in WebShell #1

In []:

```
# Set variables only for this course, locally you wont need it  
QT_X11_NO_MITSHM=1  
echo $QT_X11_NO_MITSHM  
# Start yolo V2-tiny  
roslaunch my_object_recognition_pkg yolo_v2_tiny.launch
```

NOTE: The first launch may result in a **GTK error**. [DON'T PANIC](#). Relaunch and it should work perfectly.

Move the PR2 robot around to see how it detects the different objects.

Execute in WebShell #2

In []:

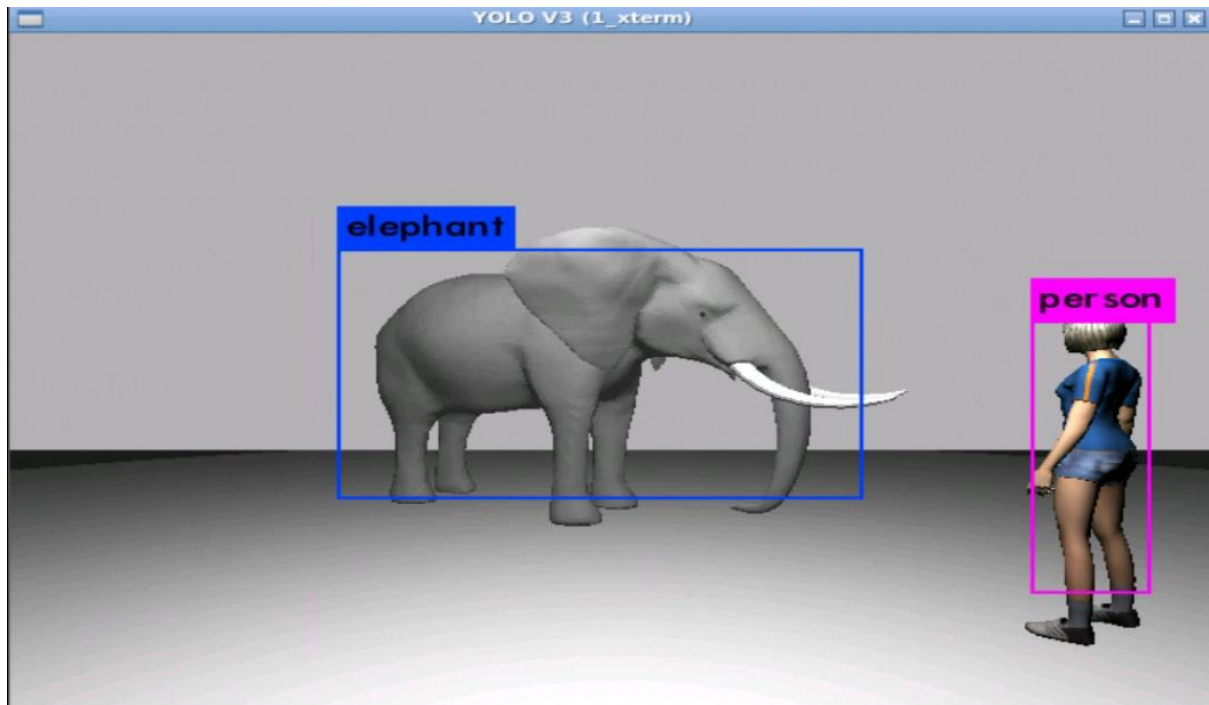
```
roslaunch pr2_tc_teleop keyboard_teleop.launch
```

Click on the Graphical Interface icon to see the detection for the YOLO system:



You will see that in this unit, **RViz is not pre-launched**. Launch RViz in a new terminal. Note, it's not necessary in this example.

You should now see something similar to the image below:



So far that was 2D

3.5 3D Object Detection and Location with YOLO

This [Darknet ROS 3D](#), created by [Francisco Martin](#), used darknet_ros in conjunction with **point-cloud data** to match the detections to 3D space and, therefore, locate more or less where the detected object is in 3D space.

This is vital for **grasping, autonomous navigation**, and many other applications.

The version used in this course was moded to work in **noetic**: [YOLO ROS TheConstruct](#).

Execute in Shell #1

In []:

```
roscd my_object_recognition_pkg
touch config/darknet_3d.yaml
touch launch/darknet_ros_3d.launch
```

SO MUCH NEEDS TO BE DONE ON EDITING THESE 2 FILES AND TESTING

- darknet_ros_3d.launch -

In []:

<launch>

```
<!-- Config camera image topic -->
<arg name="camera_rgb_topic" default="/camera/rgb/image_raw" />

<!-- Console launch prefix -->
<arg name="launch_prefix" default=""/>

<!-- Config and weights folder. -->
```

```

    <arg name="yolo_weights_path"          default="$(find
darknet_ros)/yolo_network_config/weights"/>
    <arg name="yolo_config_path"          default="$(find
darknet_ros)/yolo_network_config/cfg"/>

    <!-- ROS and network parameter files -->
    <arg name="ros_param_file"            default="$(find
darknet_ros)/config/ros.yaml"/>
    <arg name="network_param_file"        default="$(find
darknet_ros)/config/yolov2-tiny.yaml"/>

    <!-- Load parameters -->
    <rosparam command="load" ns="darknet_ros" file="$(arg
network_param_file)"/>
    <rosparam command="load" file="$(find darknet_ros)/config/ros.yaml"/>
    <param name="darknet_ros/subscribers/camera_reading/topic" type="string"
value="$(arg camera_rgb_topic)" />

    <!-- Start darknet and ros wrapper -->
    <node pkg="darknet_ros" type="darknet_ros" name="darknet_ros"
output="screen" launch-prefix="$(arg launch_prefix)">

        <param name="weights_path"          value="$(arg yolo_weights_path)" />
        <param name="config_path"           value="$(arg yolo_config_path)" />
    </node>

    <!-- Start darknet ros 3d -->
    <node pkg="darknet_ros_3d" type="darknet3d_node" name="darknet_3d"
output="screen">
        <rosparam command="load" file="$(find
my_object_recognition_pkg)/config/darknet_3d.yaml" />
    </node>
</launch>

```

```

- darknet_ros_3d.launch -
- darknet_3d.yaml -

```

In []:

```

darknet_ros_topic: /darknet_ros/bounding_boxes
output_bbx3d_topic: /darknet_ros_3d/bounding_boxes
point_cloud_topic: /camera/depth_registered/points
working_frame: camera_rgb_optical_frame
mininum_detection_thereshold: 0.3
minimum_probability: 0.3
interested_classes: ["person", "elephant", "horse", "bottle", "toothbrush",
"traffic light", "spoon", "scissors"]

```

```

- darknet_3d.yaml -

```

Comments in certain elements are important to adapt it to any robot or ROS system:

In []:

```
<arg name="camera_rgb_topic" default="/camera/rgb/image_raw" />
```

We need to set the input **RGB raw camera**.

In []:

```
<arg name="network_param_file" default="$(find
darknet_ros)/config/yolov2-tiny.yaml"/>
```

Here, we will again set the **v2 Tiny**. It's fast, especially if you want to see point-clouds in RViz (which will have a notable effect on your system load). It's the best option.

In []:

```
<rosparam command="load" file="$(find
my_object_recognition_pkg)/config/darknet_3d.yaml" />
```

Load the parameters for the **darknet_3d**. Set the following:

- **point_cloud_topic**: This is the ROS topic where your robot is publishing the point-cloud data. This is used to match the RGB detection in 2D and the point-cloud data in 3D to generate the bounding boxes in 3D.
- **minimum_probability** and **minimum_detection_threshold**: We set what we consider a valid detection and generate the bounding boxes.
- **interested_classes**: These are the objects that you want to search for. The lower the better because of the system load impact. Here we have set **people** and **elephant**.

But how do we know which object **YOLO v2 Tiny** can detect? Check the file below:

Execute in WebShell #1

In []:

```
roscd darknet_ros/config
cat yolov2-tiny.yaml
```

Start the launch and see the output:

Execute in WebShell #1

In []:

```
# Set variables only for this course, locally you wont need it
QT_X11_NO_MITSHM=1
echo $QT_X11_NO_MITSHM
# Start yolo V3
roslaunch my_object_recognition_pkg darknet_ros_3d.launch
```

You can now open **RViz**, adding a **RobotModel**, **PointCloud2** and **MarkerArray** (for visualising the detections). You can also load it from the [Perception Course Solutions Git](#), in **/perception_unit3_solutions/my_object_recognition_pkg/rviz/yolo.rviz**.

Execute in WebShell #2

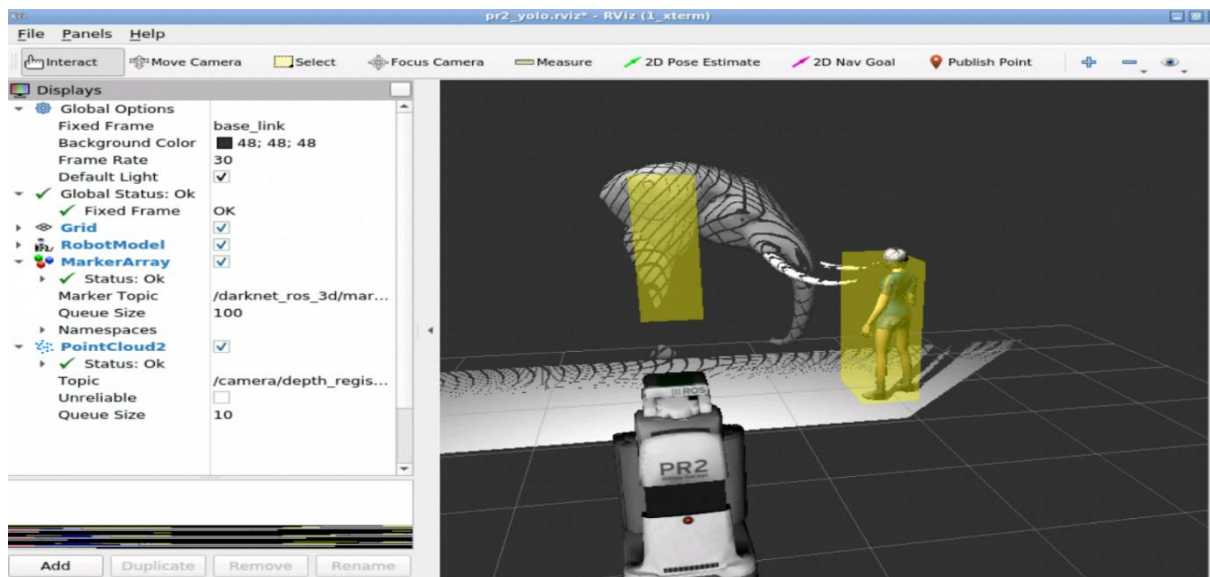
In []:

```
rviz
```

Click on the Graphical Interface icon to open the Graphical Tools:



See below the configuration you should have:



- Exercise 3.5.1 -

- **WHAT THE CODE DOES**
- Create a Python script called **yolo_3d_data_extraction.py** that extracts the marker data generated by the **darknet_ros_3d.launch** and filters only the object you want. In this case, **elephant**.
- It has to be able to store multiple objects of the same type (try with **person**).
- The topic that extracts the data is **/darknet_ros_3d/bounding_boxes**.
- You have to calculate the center of the bounding box based on the max and min values.

Remember to make the Python script executable in the Python script so that ROS can execute it:

Execute in Shell #1

In []:

```
roscd my_object_recognition_pkg
touch scripts/yolo_3d_data_extraction.py
chmod +x yolo_3d_data_extraction.py
```

- Exercise 3.5.1 -

Please attempt to complete the exercise by yourself.

Solutions for this perception course can be found in the following Git: [Perception Course Solutions Git](#)

We strongly suggest that you DO NOT USE the solutions provided unless you're absolutely stuck. You will learn much more by completing the exercises yourself.

Execute in Shell #1

In []:

```
roscd; cd ../src
git clone -b noetic
https://bitbucket.org/theconstructcore/perception_course_solutions.git
mv perception_course_solutions/perception_unit3_solutions ./
# We have to remove the rest because they might have dependencies that in
this unit are not installed
rm -rf perception_course_solutions
```



robotignite

A C A D E M Y

- Solution for Exercise 3.2.1 -

my_object_recognition_pkg/scripts/yolo_3d_data_extraction.py

In []:

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

import rospy
from gb_visual_detection_3d_msgs.msg import BoundingBoxes3d

class Yolo3DFilter:

    def __init__(self, object_name_to_search, accepted_probability=0.2):

        self._rate = rospy.Rate(5)

        self._object_name_to_search = object_name_to_search
        self._accepted_probability = accepted_probability
        self.objects_dict = {}
        self.yolo_3d_topic = "/darknet_ros_3d/bounding_boxes"
        self._check_yolo_3d_ready()
        rospy.Subscriber(self.yolo_3d_topic, BoundingBoxes3d,
self.yolo_3d_clb)

        rospy.loginfo('Ready to detect with Yolo!')

    def _check_yolo_3d_ready(self):
        yolo_3d_data = None
        while yolo_3d_data is None and not rospy.is_shutdown():
            try:
                yolo_3d_data = rospy.wait_for_message(self.yolo_3d_topic,
BoundingBoxes3d, timeout=1.0)
                rospy.logdebug("Current "+self.yolo_3d_topic+" READY=>" +
str(yolo_3d_data))

            except:
                rospy.logerr("Current "+self.yolo_3d_topic+" not ready yet,
retrying.")
```

```

def update_object_name_to_search(self, new_name):
    self._object_name_to_search = new_name

def calculate_center(self, box_data):

    x_center = (box_data.xmin + box_data.xmin) / 2.0
    y_center = (box_data.ymin + box_data.ymin) / 2.0
    z_center = (box_data.zmin + box_data.zmin) / 2.0

    return [x_center, y_center, z_center]

def yolo_3d_clb(self, msg):

    # We clean the dict
    self.objects_dict = {}
    detect_object_index = 0

    detection_boxes_array = msg.bounding_boxes

    for box in detection_boxes_array:
        object_name = box.Class
        detection_probability = box.probability
        if object_name == self._object_name_to_search:
            if detection_probability >= self._accepted_probability:
                center_array = self.calculate_center(box)
                unique_object_name = object_name+str(detect_object_index)
                self.objects_dict[unique_object_name] = center_array

                detect_object_index += 1
            else:
                rospy.logdebug("Probability too
low==" +str(detection_probability)+"<" +str(self._accepted_probability))
        else:
            rospy.logdebug("Object name doesn't
match=" +str(object_name)+", " +str(self._object_name_to_search))

def get_objects_dict_detected(self):
    return self.objects_dict

def run(self):

    while not rospy.is_shutdown():
        searched_for_objects_detected = self.get_objects_dict_detected()
        rospy.loginfo(str(searched_for_objects_detected))
        self._rate.sleep()

```

```

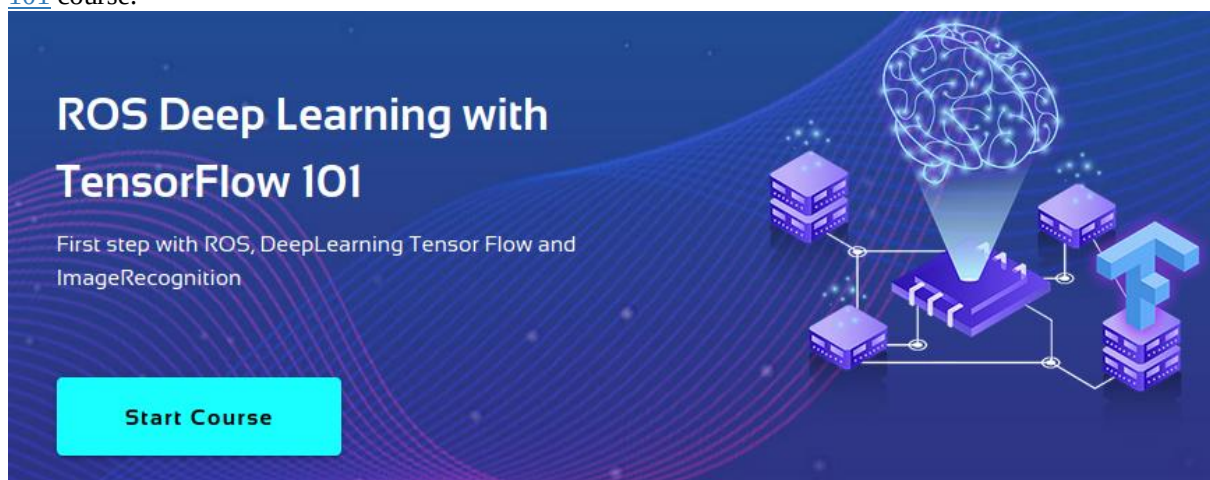
if __name__ == '__main__':
    rospy.init_node('search_for_object_node', log_level=rospy.INFO)
    yolo_obj = Yolo3DFilter(object_name_to_search="person")

    try:
        yolo_obj.run()
    except KeyboardInterrupt:
        rospy.loginfo('Shutting down')

```

- Solution for Exercise 3.2.1 -

If you want to learn more about **machine learning object detection**, check out the [TensorFlow 101](#) course:



Congratulations! You now know how to recognize objects using YOLO. In the next chapter, you will learn how to work with people-related perception.

****Project****

Select the project unit for this course. You can now do the fourth project exercise.

****END Project****